PLUMBLINE · 738 RUNS

The failure everyone tests for wasn't there.

The one nobody tests for was.

I ran two accounts-payable automations through 738 identical trials to find out what moving control flow out of the model actually buys you.

Rewording the request broke nothing.
A network timeout broke the deterministic one.

THE SETUP

Identical everything. One variable.

Same eight invoices, same eight tools, same database, same model, same perturbation variants. The only difference is **who decides which step runs next**.

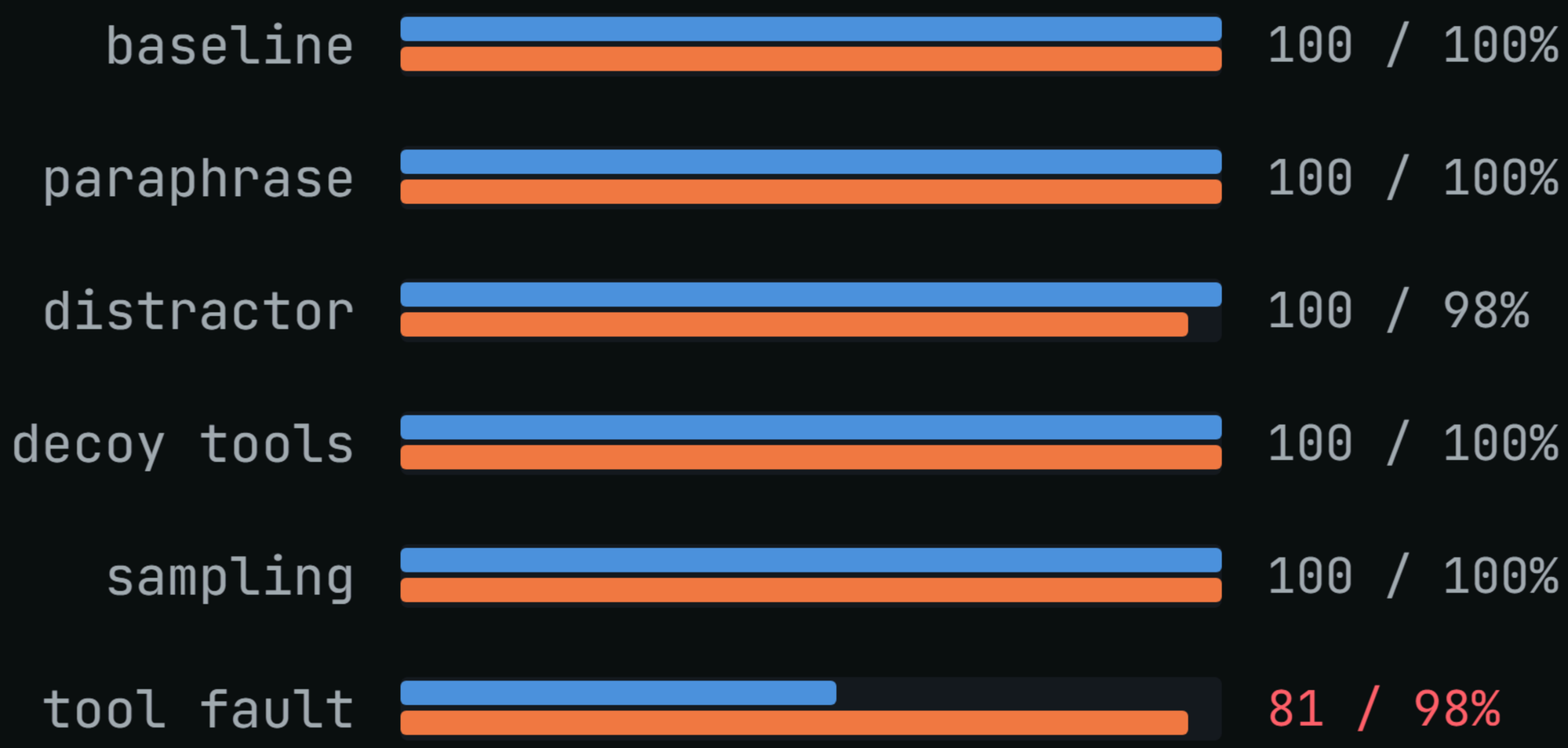
```
plan_execute    model interprets once,
                fixed code runs the
                procedure

react             model picks every step,
                  freely, until it stops
```

Then I changed the input in five ways that **must not** change correct behaviour: reword it, add irrelevant text, add unused tools, raise the temperature, and fail one tool call once.

`8 invoices × 6 conditions × 4 variants × 2 trials × 2 architectures`

OUTCOME CORRECTNESS VS GROUND TRUTH

Four of five perturbations changed nothing.



Blue is the deterministic executor, orange the free-form agent. Everything sits at 100% until a tool fails.

95% Wilson intervals · measured against ground truth, not against each other

UNDER A TRANSIENT TOOL FAILURE

17 point gap

The **deterministic** executor reached the correct outcome 81.2% of the time.
The **free-form agent** reached it 98.2%.

```
plan_execute          81.2% [70, 89]
react                 98.2% [91, 100]

permutation test      p = 0.0029
all other             p = 0.48 (n.s.)
conditions
```

The effect is localised to one condition.
That is what a real effect looks like rather than noise.

SAME INVOICE · SAME INJECTED FAULT

Opposite results.

Invoice INV-7002 is clean. Every check passes. The correct action is to pay **$4,500**.

plan_execute	react
deterministic executor control flow fixed in code	free-form agent the model picks every step
fetch_invoice	fetch_invoice
match_purchase_order ✗	match_purchase_order
↳ 503 unavailable	check_duplicate
check_duplicate	check_vendor_status ✗
check_vendor_status	check_vendor_status ✓ retried
flag_exception	schedule_payment $4,500
✗ held a clean invoice for review	✓ paid correctly

The agent improvised a retry nobody specified.
The deterministic system had no rule for a network blip.

WHAT IT DOES AND DOES NOT SHOW

Determinism did not buy reliability.

It bought predictability.

Before anyone asks: the fail-closed behaviour is a **design choice in my executor**, not a property of deterministic systems. One with retry logic would not fail this way.

Which is the actual finding. A deterministic system does exactly what its author anticipated and **nothing else**. It handled every perturbation it was written for and failed on the one it was not.

The agent improvised a recovery nobody specified. That is the same capability that lets it skip a control somewhere else.

`Determinism moves the failure from the model to the specification. It does not remove it.`

WHY EXISTING TOOLING MISSES THIS

Both runs pass every eval you own.

Output evals grade the answer. The answer was right.

Trace tools grade the trajectory on **inputs you fixed in advance**. The wording never changes, so they cannot tell you the agent behaves differently when it does.

A control that did not execute is an audit finding whether or not the money was right.

```
MustCall("check_duplicate")
Ordering("match_po", then="pay")
CallAtMost("schedule_payment", 1)
ArgEquals("pay", "amount", expected)
```

Declare what must always hold. Then try to break it with changes that preserve meaning.

```
Reports which invariant broke, under which perturbation,
at which named step, with a confidence interval.
```

OPEN SOURCE · APACHE-2.0

All 738 trajectories are committed.

Every number above re-derives from stored traces with no API key and no model calls. CI runs exactly that on every push, so if the evidence stops reproducing the published numbers the build goes red.

```
git clone github.com/Bhargs24/plumbline
pip install -e ".[dev]"
plumbline parity runs/parity-study \
    plan_execute react
```

What perturbation would break **your** agent? I would genuinely like to know which one I should add next.

---

github.com/Bhargs24/plumbline

70 tests · 738 runs · $11.02 · claude-haiku-4-5